

CA PracticeOS Compliance — AI Agent Layer

AICA Level 2 Capstone Project Summary
Group B839-15 · Shubham Mishra, Shiv Kumar Sharma, Shrey Chopra, Sonali Pawar · September 2026

The Problem

A CA firm's practice-management work is document-heavy and largely manual: client details get retyped from PAN/GST documents, engagement letters and certificates are drafted from scratch each time, and checking a drafted document against the mandatory clauses a professional standard requires is a manual, easy-to-skip step. Our Level 1 project (CA PracticeOS Compliance) built a browser-based document engine — a Client Master, a document drafting utility, and procedural checklists — but it had no reasoning layer: it could not read a document and tell you whether it was actually compliant.

The Solution

This project adds a real, multi-step reasoning AI agent layer on top of the existing L1 application, built around three agents and one third-party connector:

- **Agent 3 — Compliance Checklist Agent (the headline feature).** Reads a drafted document and checks it against a mandatory-clause checklist for its document type (six types, derived from the L1 app's own templates), as a perceive → reason → self-check → act loop: one Claude call verdicts every clause with quoted evidence, then an asymmetric keyword self-check catches the model over-claiming a clause is present. Output: a pass/fail verdict with exactly what's missing.
- **Agent 1 — Client Onboarding Agent.** Turns a photo/PDF of a PAN card or GST certificate into a Client Master row matching the L1 app's exact import schema (validated against PAN/GSTIN format and cross-checked against each other), ready to import via the app's existing bulk-upload feature.
- **Agent 2 — Document Drafting Agent.** Turns one plain-English instruction (e.g. "Prepare an engagement letter for ABC Pvt Ltd for tax audit AY 2025-26, fees Rs 50,000") into a filled draft, reusing the L1 app's own document structure and firm defaults, then feeds that draft straight into Agent 3.
- **Tally Connector.** Connects via ICAI's Tally Prime MCP Server behind a reusable BaseConnector interface, so Zoho Books and SAP could be added later without touching the rest of the backend (they are explicitly not built in this timeline).

Architecture

Layer	Implementation
Backend	Python 3.14, FastAPI (src/main.py) — 8 endpoints across all 3 agents
LLM	Claude API (claude-sonnet-5) — one call per agent step, chained
Demo UI	Streamlit — all 3 agents + the Tally connector in one app
Connector	BaseConnector interface; TallyConnector via ICAI Tally MCP Server (stdio)
Document parsing	pdfplumber for text-layer PDFs; Claude vision for images and scanned PDFs
Bridge to L1	Agent 1 exports the exact CSV schema the L1 app's Client Master import expects

L1 → L2 Evolution

L1 built the document engine: a Client Master, document generators (engagement letters, certificates, fee notes), and procedural checklists, all running client-side in the browser. L2 makes it intelligent and connected: a real backend, LLM-driven extraction and reasoning, a validating compliance agent, and a path to live practice-management data via Tally. The two are built to interoperate directly — Agent 1's CSV output was tested live against the actual L1 app's import feature, not just checked against its schema on paper.

Key Proof Points

- Agent 1 → L1 import proven end to end in a real browser session: a generated CSV was fed through the L1 app's actual "Upload / Update Client Master" flow and the client appeared correctly in the table.
- Agent 3 caught a genuine gap in the L1 app's own engagement letter template — not a synthetic test case. Feeding Agent 2's draft (built by deliberately reusing the L1 template as-is) into Agent 3 produces a real FAIL for a missing Limitations clause, because that clause was never in the original template.
- The Tally connector is honest about a real infrastructure constraint: Tally's MCP server is a local process, unreachable from a cloud deployment, so every connector result is explicitly tagged "source": "live" or "cached_fallback" rather than silently faking live data.

Learnings

A validating agent needs a genuinely independent second opinion, not just a JSON wrapper around one model call — but that second opinion has to be asymmetric: a keyword heuristic is reliable for catching an LLM over-claiming a clause is present, but unreliable for second-guessing an "absent" verdict, since judging whether placeholder text under an on-topic heading counts as real content needs the same semantic judgment only the LLM can make. We also learned to verify third-party integrations against their real documented architecture before writing a connector, rather than assuming a cloud-friendly API existed — building around Tally's actual local-only MCP design, and disclosing that constraint honestly, produced a more credible result than pretending otherwise.

Roadmap (Not Built in This Timeline)

Zoho Books and SAP connectors, designed to extend to via the same BaseConnector interface. Full cloud deployment to Render (configuration is complete in render.yaml; awaiting GitHub/Render account setup).